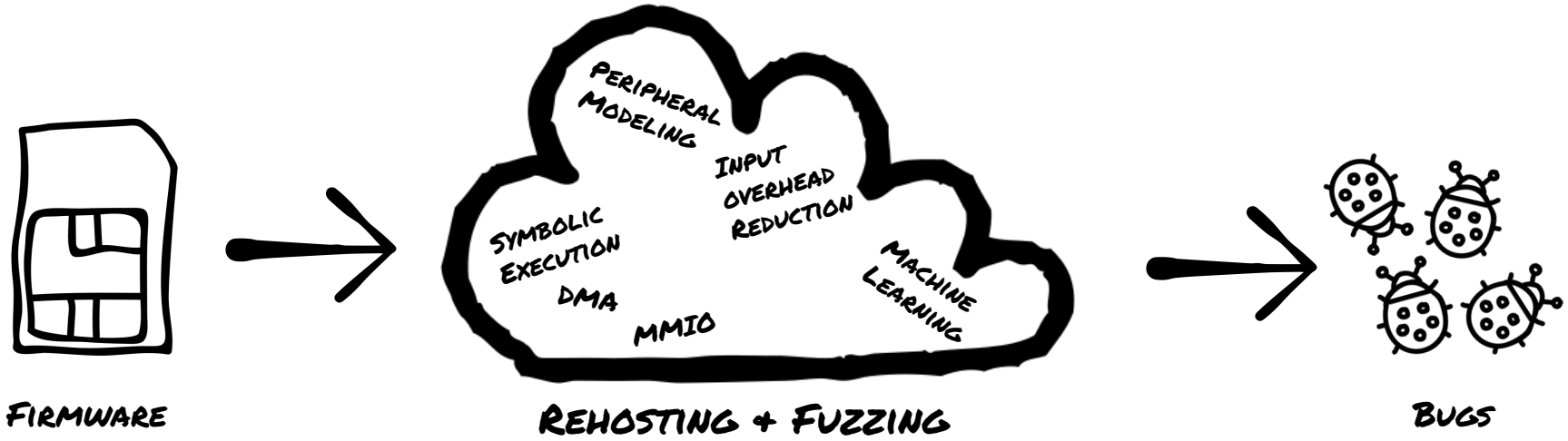
Defending Firmware

Secure Software & Hardware Systems

Overview

- Recap Rehosting
- Challenges for defending firmware
  - Design constraints
  - Fragmented development process
  - Missing entropy sources
- Traditional defenses & firmware
  - Non-executable memory
  - Address space layout randomization
  - Stack canaries
  - Adoption
- Additional Defense Examples
  - Static binary sanitization
  - Return address integrity
  - Hotpatching
- Recap

What do we want?



Rehosting

“Firmware re-hosting is the process of migrating firmware from its original hardware environment into a virtual environment”

Harrison, Lee, et al. "PARTEMU: Enabling Dynamic Analysis of Real-World TrustZone Software Using Emulation." *USENIX Security’20*

Hardware-in-the-Loop Rehosting: Approaches

Snapshot + Transfer

- Goal: Remove hardware accesses from emulation
- How: Identify IO-heavy parts and execute on-device, then transfer state
- Caveat: Later hardware interactions are problematic

Remote Memory

- Goal: Forward MMIO accesses to physical device
- How: Emulation hooks and on-device debugging stubs
- Caveat: Performance, timing-sensitive operations

Hardware-less Rehosting: Approaches

Using Abstractions

- Goal: Avoid hardware interactions
- How: Identify & hook interfaces of OS/libraries
- Caveat: Interfaces not always known

Modeling Hardware

- Goal: Model hardware “well enough”
- How: Heuristics, Program Analysis
- Caveat: Difficult to scale to complex peripherals

Overview

- Recap Rehosting
- Challenges for defending firmware
  - Design constraints
  - Fragmented development process
  - Missing entropy sources
- Traditional defenses & firmware
  - Non-executable memory
  - Address space layout randomization
  - Stack canaries
  - Adoption
- Additional Defense Examples
  - Static binary sanitization
  - Return address integrity
  - Hotpatching
- Recap

Recent Research

Securing Real-Time Microcontroller Systems through Customized Memory View Switching

Chung Hwan Kim*§ Taegyu Kim† Hongjun Choi† Zhongshu Gu‡
Byoungyoung Lee† Xiangyu Zhang† Dongyan Xu†
*NEC Laboratories America †Purdue University ‡IBM T.J. Watson Research Center
*chungkim@nec-labs.com †{tgkim, choi293, byoungyoung, xyzhang, dxu}@p

Protecting Bare-metal Embedded Systems With Privilege Overlays

Abraham A. Clements*, Naif Saleh Almakhdhub†, Khaled S. Saab‡, Prashast Srivastava†,
Jinkyu Koo†, Saurabh Bagchi†, Mathias Payer†
*Purdue University and Sandia National Laboratories, clemen19@purdue.edu
†Purdue University, {nalmakhd, srivas41, kooj, sbagchi}@purdue.edu, mathias.payer@nebelwelt.net
‡Georgia Institute of Technology, ksaab3@purdue.edu

RapidPatch: Firmware Hotpatching for Real-Time Embedded Devices

Yi He, Zhenhua Zou — *Tsinghua University and BNRist*
Kun Sun — *George Mason University*
Zhuotao Liu, Ke Xu — *Tsinghua University and BNRist*
Qian Wang — *Wuhan University*
Chao Shen — *Xi'an Jiaotong University*
Zhi Wang — *Florida State University*
Qi Li — *Tsinghua University and BNRist*

HERA: Hotpatching of Embedded Real-time Applications

Christian Niesler, Sebastian Surminski, Lucas Davi
University of Duisburg-Essen, Germany
{christian.niesler, sebastian.surminski, lucas.davi}@uni-due.de

OAT: Attesting Operation Integrity of Embedded Devices

Zhichuang Sun — Northeastern University
Bo Feng — Northeastern University
Long Lu — Northeastern University
Somesh Jha — University of Wisconsin

μRAI: Securing Embedded Systems with Return Address Integrity

Naif Saleh Almakhdhub — Purdue University and King Saud University — nalmakhd@purdue.edu
Abraham A. Clements — Sandia National Laboratories — aacleme@sandia.gov
Saurabh Bagchi — Purdue University — sbagchi@purdue.edu
Mathias Payer — EPFL — mathias.payer@nebelwelt.net

μXOM: Efficient eXecute-Only Memory on ARM Cortex-M

Donghyun Kwon[1,2] Jangseop Shin[1,2] Giyeol Kim[1,2]
Byoungyoung Lee[1,3] Yeongpil Cho[4] Yunheung Paek[1,2]
[1]*ECE, Seoul National University,* [2]*ISRC, Seoul National University*
[3]*Computer Science, Purdue University,* [4]*School of Software, Soongsil University*

Silhouette: Efficient Protected Shadow Stacks for Embedded Systems

Jie Zhou[1], Yufei Du[1], Zhuojia Shen[1], Lele Ma[1,2], John Criswell[1], and Robert J. Walls[3]
[1]University of Rochester
[2]College of William & Mary
[3]Worcester Polytechnic Institute

IRQDebloat: Reducing Driver Attack Surface in Embedded Devices

Zhenghao Hu — New York University — huzh@nyu.edu
Brendan Dolan-Gavitt — New York University — brendandg@nyu.edu

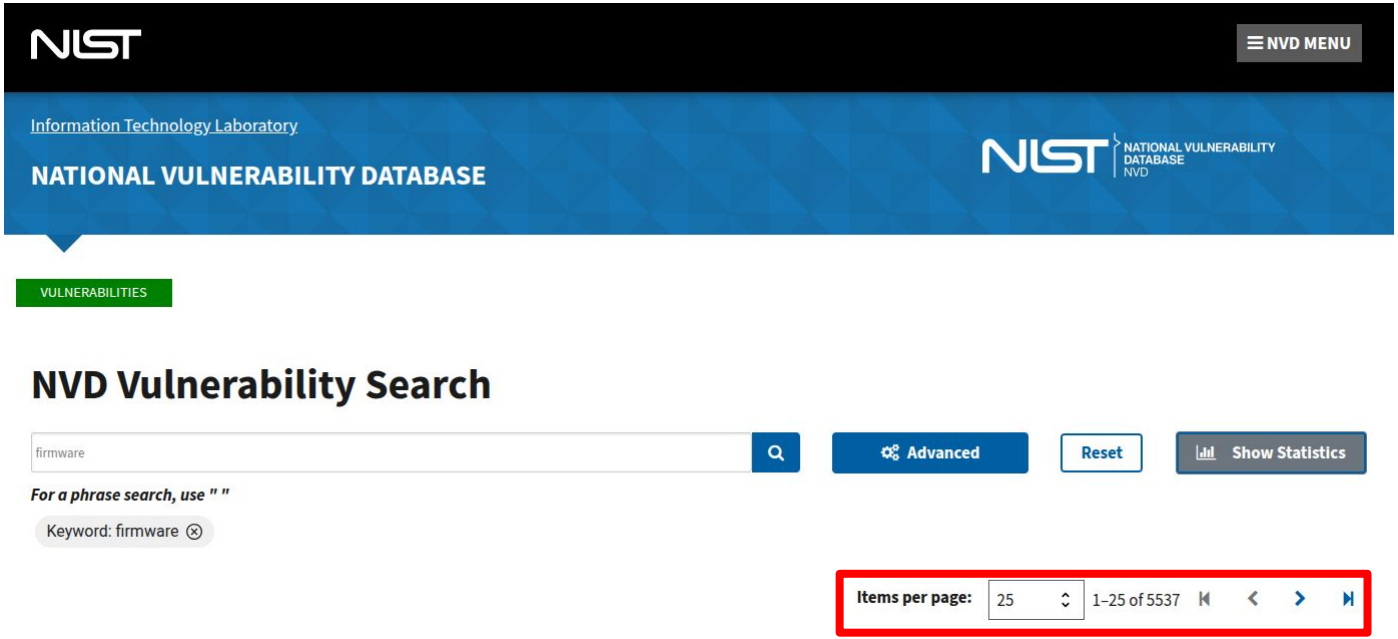
μSBS: Static Binary Sanitization of Bare-metal Embedded Devices for Fault Observability

Majid Salehi — *imec-DistriNet, KU Leuven* — *majid.salehi@kuleuven.be*
Danny Hughes — *imec-DistriNet, KU Leuven* — *danny.hughes@kuleuven.be*
Bruno Crispo — *imec-DistriNet, KU Leuv* — *Trento University, Italy* — *bruno.crispo@unitn.it*

MicroGuard: Securing Bare-Metal Microcontrollers against Code-Reuse Attacks

Majid Salehi — *imec-DistriNet, KU Leuven* — majid.salehi@cs.kuleuven.be
Danny Hughes — *imec-DistriNet, KU Leuven* — danny.hughes@cs.kuleuven.be
Bruno Crispo — *imec-DistriNet, KU Leuven* — *Trento University, Italy* — bruno.crispo@cs.kuleuven.be

Reality



https://nvd.nist.gov/vuln/search

Identifier	CISA Kev Info	Published Date	CNA	Description
CVE-2025-9725	✖	2025-08-31	VulDB	A vulnerability was identified in Cudy LT500E up to 2.3.12. Affected is an unknown function of the file /squashfs-root/etc/shadow of the component Web Interface. The manipulation leads to use of hard-coded password. The attack must be carried out locally. The attack's complexity is rated as high. Th...
CVE-2025-9696	✖	2025-09-02	ICS-CERT	The SunPower PVS6's BluetoothLE interface is vulnerable due to its use of hardcoded encryption parameters and publicly accessible protocol details. An attacker within Bluetooth range could exploit this vulnerability to gain full access to the device's servicing interface. This access allows the atta...
CVE-2025-9574	✖	2025-10-20	Asea Brown Boveri Ltd. (ABB)	Missing Authentication for Critical Function vulnerability in ABB ALS-mini-s4 IP, ABB ALS-mini-s8 IP.This issue affects . All firmware versions with the Serial Number from 2000 to 5166
CVE-2025-9380	✖	2025-08-24	VulDB	A vulnerability was identified in FNKvision Y215 CCTV Camera 10.194.120.40. Affected by this issue is some unknown functionality of the file /etc/passwd of the component Firmware. Such manipulation leads to hard-coded credentials. Local access is required to approach this attack. The exploit is publ...
CVE-2025-9379	✖	2025-08-24	VulDB	A vulnerability was determined in Belkin AX1800 1.1.00.016. Affected by this vulnerability is an unknown functionality of the component Firmware Update Handler. This manipulation causes insufficient verification of data authenticity. The attack can be initiated remotely. The vendor was contacted ear...
CVE-2025-9265	✖	2025-10-13	Switzerland Government Common Vulnerability Program	A broken authorization vulnerability in Kiloview NDI N30 allows a remote unauthenticated attacker to deactivate user verification, giving them access to state changing actions that should only be initiated by administratorsThis issue affects Kiloview NDI N30 and was fixed in Firmware version lat...
CVE-2025-9195	✖	2025-08-28	Solidigm	Improper input validation in firmware of some Solidigm DC Products may allow an attacker with local access to cause a Denial of Service
CVE-2025-9146	✖	2025-08-19	VulDB	A flaw has been found in Linksys E5600 1.1.0.26. The affected element is the function verify_gemtek_header of the file checkFw.sh of the component Firmware Handler. Executing manipulation can lead to risky cryptographic algorithm. The attack may be launched remotely. The attack requires a high level...
CVE-2025-9133	✖	2025-10-21	Zyxel Corporation	A missing authorization vulnerability in Zyxel ATP series firmware versions from V4.32 through V5.40, USG FLEX series firmware versions from V4.50 through V5.40, USG FLEX 50(W) series firmware versions from V4.16 through V5.40, and USG20(W)-VPN series firmware versions from V4.16 through V5.40 could...
CVE-2025-8980	✖	2025-08-14	VulDB	A vulnerability has been found in Tenda G1 16.01.7.8(3660). Affected by this issue is the function check_upload_file of the component Firmware Update Handler. The manipulation leads to insufficient verification of data authenticity. The attack may be launched remotely. The complexity of an attack is...
CVE-2025-8979	✖	2025-08-14	VulDB	A vulnerability was identified in Tenda AC15 15.13.07.13. Affected by this vulnerability is the function check_fw_type/split_fireware/check_fw of the component Firmware Update Handler. The manipulation leads to insufficient verification of data authenticity. The attack can be launched remotely. The ...
CVE-2025-8978	✖	2025-08-14	VulDB	A vulnerability was determined in D-Link DIR-619L 6.02CN02. Affected is the function FirmwareUpgrade of the component boa. The manipulation leads to insufficient verification of data authenticity. It is possible to launch the attack remotely. The complexity of an attack is rather high. The exploitab...
CVE-2025-8915	✖	2025-10-13	Switzerland Government Common Vulnerability Program	Hardcoded TLS private key and certificate in firmware in Kiloview N30 2.02.246 allows malicious adversary to do a Mann-in-the-middle attack via the network
CVE-2025-8731	✖	2025-08-08	VulDB	A vulnerability was identified in TRENDnet TI-G160i, TI-PG102i and TPL-430AP up to 20250724. This affects an unknown part of the component SSH Service. The manipulation leads to use of default credentials. It is possible to initiate the attack remotely. The exploit has been disclosed to the public a...

Reality

- Plenty of matches!
  - Over ⅕ with high & critical severity!
- Different device types, e.g.:
  - Routers
  - Printers
  - BIOS
  - Network interface
  - Unnamed IoT devices
- Different CVE Numbering Authorities (CNAs)

This lecture

- Point out problems for defending firmware
- Investigate the applicability of desktop defenses for firmware
- Discuss exemplary recent defenses

Overview

- Recap Rehosting
- Challenges for defending firmware
  - Design constraints
  - Fragmented development process
  - Missing entropy sources
- Traditional defenses & firmware
  - Non-executable memory
  - Address space layout randomization
  - Stack canaries
  - Adoption
- Additional Defense Examples
  - Static binary sanitization
  - Return address integrity
  - Hotpatching
- Recap

Difficulties of defending firmware (1)

- Design constraints:
  - Power consumption
  - Processing power
  - ROM & RAM size
  - Real-time assurances
- Effects:
  - “Low-hanging fruit” defenses may be omitted
  - Unclear error handling strategies

Difficulties of defending firmware (2)

- Fragmented development process
  - Chip vendor
  - Original equipment manufacturer (OEM)
  - Product manufacturer
  - Optional: OS developers
- Effects:
  - Unclear security guarantees
  - Difficult patching process
  - Reporting nightmare

Difficulties of defending firmware (3)

- Missing entropy sources
  - Generating true random numbers is difficult
  - Even (secure) PRNGs need a seed
  - Non-deterministic input sources may be missing

- Effects:
  - Cryptographic assumptions may not hold
  - Security mechanisms reliant on entropy are bypassable

Overview

- Recap Rehosting
- Challenges for defending firmware
  - Design constraints
  - Fragmented development process
  - Missing entropy sources
- Traditional defenses & firmware
  - Non-executable memory
  - Address space layout randomization
  - Stack canaries
  - Adoption
- Additional Defense Examples
  - Static binary sanitization
  - Return address integrity
  - Hotpatching
- Recap

Traditional Desktop Defenses

Desktop systems deploy common set of defenses:

- Non-Executable Memory
- Address Space Layout Randomization
- Stack canaries
- Heap hardening
- And more!

Can we apply them to firmware?

Traditional Desktop Defenses

Desktop systems deploy common set of defenses:

- Non-Executable Memory
- Address Space Layout Randomization
- Stack canaries
- ~~Heap hardening~~
- ~~And more!~~

Can we apply them to firmware?

Non-Executable Memory

- Standard feature in modern MMUs
  - But embedded systems may not have MMU!
- Embedded “solution”: Memory Protection Unit
  - No virtual memory
  - Basic memory protection features
  - For ARM: widely available since ARM Cortex-M
- Questionable adoption:

> “Use of an MPU necessarily makes application design more complex because first the MPU’s memory region restrictions must be determined and described to the RTOS, and second the MPU restricts what application tasks can and cannot do.”
> − https://www.freertos.org/FreeRTOS-MPU-memory-protection-unit.html

Non-Executable Memory

- Implementations assume virtual address space
  - Not possible without MMU
- Firmware often lives in ROM
  - Impossible to randomize for every execution
  - Compile-time randomization exists, but does not provide same guarantees
- Physical address space is problematic
  - MMIO at fixed addresses
  - ROM & RAM as well
- Possible lack of entropy

Stack Canaries

- Increases code size, stack size, and run time
  - Direct opposition to by-design requirements

- Even if present, typically static values
  - Trivial to bypass if attacker has access to FW

- Possible lack of entropy

Adoption of Defenses

- Recent study [1] investigates adoption of defenses in embedded OSs & CPU cores

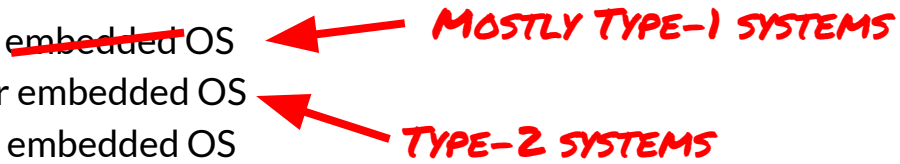
- Different classification scheme:
  - Mobile embedded OS
  - Regular embedded OS
  - Deeply embedded OS

[1] Abbasi et al. “Challenges in Designing Exploit Mitigations for Deeply Embedded Systems.” EuroS&P’19

Adoption of Defenses

- Recent study [1] investigates adoption of defenses in embedded OSs & CPU cores

- Different classification scheme:
  - Mobile ~~embedded~~ OS
  - Regular embedded OS
  - Deeply embedded OS



[1] Abbasi et al. “Challenges in Designing Exploit Mitigations for Deeply Embedded Systems.” EuroS&P’19

Adoption of Defenses: NX, ASLR & Canaries

OS	ESP	ASLR	Canarie	OS	ESP	ASLR	Canaries
BlackBerry OS	✓	✓	✓	Android*	✓	✓	✓
iOS*	✓	✓	✓	Win 10 Mob.*	✓	✓	✓
Sailfish OS*	✓	✓	✓	Tizen*	✓	✓	✓
Ubuntu Core*	✓	✓	✓	Brillo*	✓	✓	✓
Yocto Linux*	✓	✓	✓	Windows Embedded*	✓	✓	✓
OpenWRT*	✓	✓	✓	Junos OS*	✓	×	✓
μClinux*	✓	×	✓	CentOS*	✓	✓	✓
NetBSD*	✓	✓	✓	IntervalZero RTX*	✓	×	✓
ScreenOS	×	×	×	Enea OSE	×	×	×
QNX	✓	✓	✓	VxWorks	✓	×	×
INTEGRITY	✓	×	×	RedactedOS [2]	×	×	×
Cisco IOS	×	×	×	eCos	×	×	×
Zephyr	✓	×	✓	ThreadX	×	×	×
Nucleus	×	×	×	NXP MQX	×	×	×
Kadak AMX	×	×	×	Keil RTX	×	×	×
RTEMS	×	×	×	freeRTOS	×	×	×
Micrium μC/OS[1]	✓	×	×	TI-RTOS	×	×	×
DSP/BIOS	×	×	×	TinyOS	×	×	×
LiteOS	×	×	×	RIOT	×	×	×
ARM mbed	✓	×	×	Contiki	×	×	×
Nano-RK	×	×	×	Mantis	×	×	×

Image source & more information:
Abbasi et al. “Challenges in Designing Exploit Mitigations for Deeply Embedded Systems.” EuroS&P’19

Adoption of Defenses: Memory & RNGs

OS	MPROT	VMEM	RNG	OS	MPROT	VMEM	RNG
Android*	✓	✓	✓	iOS*	✓	✓	✓
Win10 Mob.*	✓	✓	✓	BlackBerry OS	✓	✓	✓
Tizen*	✓	✓	✓	Sailfish OS*	✓	✓	✓
Ubuntu Core*	✓	✓	✓	Brillo*	✓	✓	✓
Yocto Linux*	✓	✓	✓	Windows Embedded*	✓	✓	✓
OpenWRT*	✓	✓	✓	Junos OS*	✓	✓	✓
μClinux*	✓	✓	✓	CentOS*	✓	✓	✓
NetBSD*	✓	✓	✓	IntervalZero RTX*	✓	✓	✓
ScreenOS	✓	✓	✓	Enea OSE	✓	✓	×
QNX	✓	✓	✓	VxWorks	✓	✓	×
INTEGRITY	✓	✓	✓	RedactedOS	✓	✓	✓
Cisco IOS	×	×	✓	eCos	×	×	×
Zephyr	✓	×	×	ThreadX	✓	×	×
Nucleus	✓	×	×	NXP MQX	✓	×	×
Kadak AMX	×	×	×	Keil RTX	✓	×	×
RTEMS	×	×	×	FreeRTOS	✓	×	×
Micrium μC/OS	✓	×	×	TI-RTOS	✓	×	×
DSP/BIOS	✓	×	×	TinyOS	✓	×	×
LiteOS	✓	×	×	RIOT	✓	×	×
ARM mbed	✓	×	×	Contiki	×	×	×
Nano-RK	×	×	×	Mantis	×	×	×

Image source & more information:
Abbasi et al. “Challenges in Designing Exploit Mitigations for Deeply Embedded Systems.” EuroS&P’19

Adoption of Defenses: MMU & MPU

Core Family	Arch.	MPU	MMU	ESP
ARM				
ARM1	N	×	×	×
ARM2	N	×	×	×
ARM3	N	×	×	×
ARM6	N	×	×	×
ARM7	N	×	×	×
ARM7T	N	~	~	×
ARM7EJ	N	×	×	×
ARM8	N	×	✓	×
ARM9T	N	~	~	×
ARM9E	N	~	~	×
ARM10E	N	×	✓	×
ARM11	N	~	~	✓
ARM Cortex-A	N	×	✓	✓
ARM Cortex-R	N	✓	×	✓
ARM Cortex-M	N	~	×	✓
PIC				
PIC10	H	×	×	×
PIC12	H	×	×	×
PIC16	H	×	×	×
PIC18	H	×	×	×
PIC24	H	×	×	×
dsPIC	H	×	×	×

Core Family	Arch.	MPU	MMU	ESP
MIPS32				
PIC32MX	N	×	×	×
PIC32MZ EC	N	×	✓	×
PIC32MZ EF	N	×	✓	✓
PIC32MM	N	×	✓	✓
PowerPC				
PPC e200	N	~	~	✓
PPC e300	N	×	✓	✓
PPC e500	N	×	✓	✓
PPC e600	N	×	✓	✓
PPC 403	N	×	×	×
PPC 401	N	×	×	×
PPC 405	N	×	✓	✓
PPC 440	N	×	✓	✓
PPC 740	N	×	✓	✓
PPC 750	N	×	✓	✓
PPC 603	N	×	✓	✓
PPC 604	N	×	✓	✓
PPC 7400	N	×	✓	✓

Image source & more information:
Abbasi et al. “Challenges in Designing Exploit Mitigations for Deeply Embedded Systems.” EuroS&P’19

Adoption of Defenses: Interpretation

- Type-I firmware mostly deploy standard defenses
- Type-II firmware rarely deploy standard defenses
- MMU/MPU is not commonly available
  - Only ~half of analyzed cores support MPU or MMU
- No information about deployed cores & OSs

Interpretation based on data provided in:
Abbasi et al. “Challenges in Designing Exploit Mitigations for Deeply Embedded Systems.” EuroS&P’19

Additional Reading Suggestion

Tan, Xi, et al. "Where's the "up"?! A Comprehensive (bottom-up) Study on the Security of Arm Cortex-M Systems." USENIX Woot Conference on Offensive Technology (WOOT). 2024

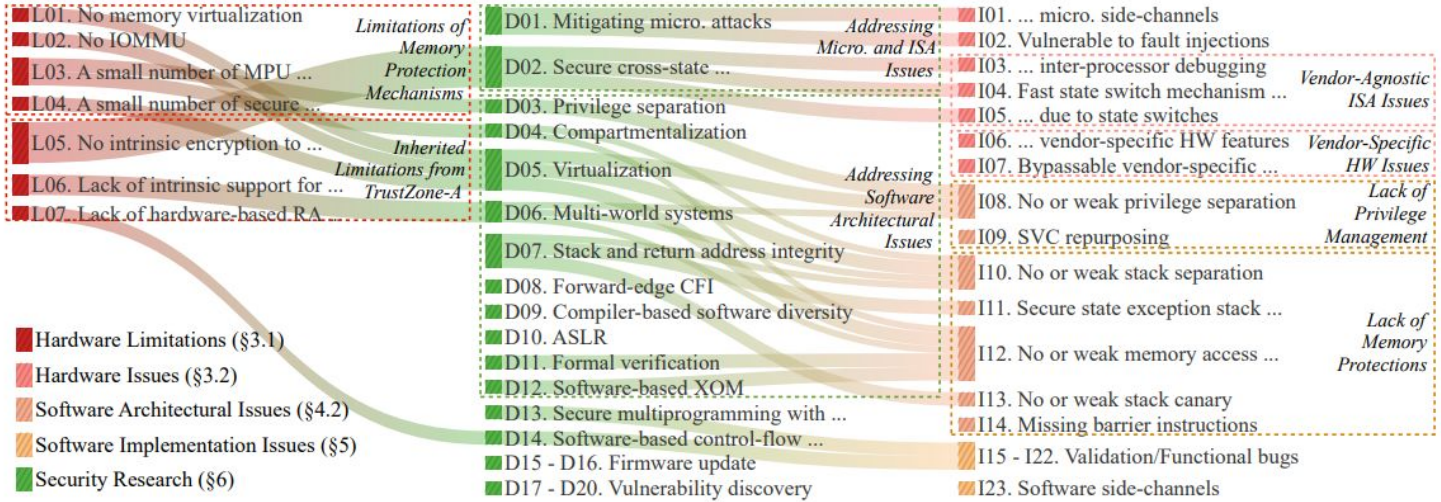


Figure 3. The relationships among the systematized Cortex-M related limitations, issues, and mitigations. The connections indicate the issues a research direction attempts to address and the limitations it needs to overcome. For instance, to address the issue of *no or weak privilege separation* (**I08**), mitigations (**D03**, **D05**, and **D06**) have been proposed, and they overcome some limitations (**L01**, **L02**, and **L03**). An interactive version of this figure can be accessed at our anonymized repo.

Overview

- Recap Rehosting
- Challenges for defending firmware
  - Design constraints
  - Fragmented development process
  - Missing entropy sources
- Traditional defenses & firmware
  - Non-executable memory
  - Address space layout randomization
  - Stack canaries
  - Adoption
- Additional Defense Examples
  - Static binary sanitization
  - Return address integrity
  - Hotpatching
- Recap

Additional Defense Examples: Static Binary Sanitization

- Static Instrumentation for sanitization
  - Goal to detect corruptions to enable fuzzing
  - Based on static disassembly & binary rewriting
  - Evaluated on Cortex-M3/4 MCUs
- Detects following memory corruptions:
  - Null Pointer Dereference, Stack-based buffer overflow, Heap-based buffer overflow, Format String, Double Free
- Average overhead:
  - Runtime: ~32.5%
  - Flash size: ~19%

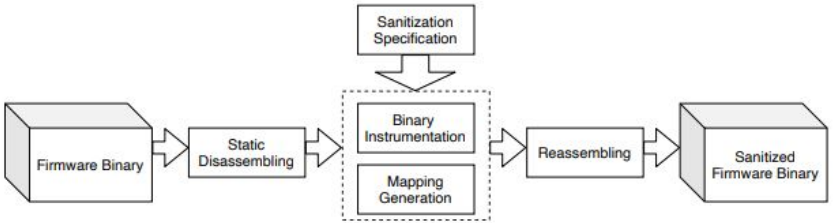
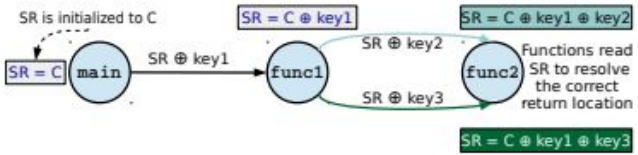


Image source & more information:
Salehi et al. “µSBS: Static Binary Sanitization of Bare-metal Embedded Devices for Fault Observability.” RAID’20

Additional Defense Examples: Return Address Integrity

- Compiler-based backward-edge CFI
  - Protection against overwriting return addresses
  - Implementation on top of LLVM
  - Evaluated on Cortex-M4 CPU



- Requires MPU and exclusive general purpose register
  - Table of possible return targets in RX memory
  - Function ID stored in exclusive register
  - Function ID generated with call site specific function key (xor)

- Average overhead:
  - Runtime: ~0.1%
  - Flash size: ~34.6%

Image source & more information:
N. S. Almakhdhub et al. “µRAI: Securing Embedded Systems with Return Address Integrity.” NDSS’20

Additional Defense Examples: Hot-Patching

- Not all firmware can be updated:
  - Some devices may need to operate continuously
  - Patched firmware could have undesired side-effects
  - Solution: Hotpatching
- HERA: A PoC implementation
  - Uses Flash Patch and Breakpoint Unit (FPB)
  - Implemented on top of FreeRTOS
  - Provides deterministic execution time
- No classic overhead measurement
  - Instead, overhead in terms of cycles and ns

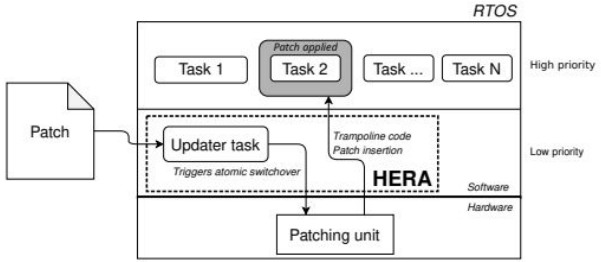


Image source & more information:
Niesler et al. “HERA: Hotpatching of Embedded Real-time Applications.” NDSS’21

Additional Defense Examples: Summary

- A lot of technical details omitted
  - If you are interested, check the papers!
- Investigated approaches:
  - Binary-based instrumentation
  - Compiler-based instrumentation
  - Hotpatching
- Typical problems:
  - Overhead
  - Source code required
  - Special hardware requirements

Overview

- Recap Rehosting
- Challenges for defending firmware
  - Design constraints
  - Fragmented development process
  - Missing entropy sources
- Traditional defenses & firmware
  - Non-executable memory
  - Address space layout randomization
  - Stack canaries
  - Adoption
- Additional Defense Examples
  - Static binary sanitization
  - Return address integrity
  - Hotpatching
- Recap

Recap

- Challenges for defending firmware
  - “Why” is defending firmware not as easy?

- Traditional desktop defenses on firmware
  - “How” do these defenses translated to firmware?
  - “How” is the adoption?

- Recent research approaches
  - “What” is currently being done?